# 多租户隔离（TiDB 无 RLS 的补偿方案）

> 背景：DECISIONS.md D23 = 全用 TiDB。TiDB 没有 PostgreSQL 的 Row Level Security。
> 因此"隔离"不能靠数据库，只能靠工程约束 + 自动化验证。 本文是唯一有效的隔离规范，违反即视为 P0 缺陷。

## 1. 为什么这是最高风险点

20 个基地 + N 个供应商共用一个库。一次漏写 `WHERE tenant_id = ?`，就是把 A 基地的设备数据给 B 基地看到——在等保三级下这是数据泄露事件，不是 bug。

PostgreSQL 的 RLS 能在数据库层兜底（即使代码写错也返回空集）。TiDB 不能。所以必须用更多层补回来。

## 2. 五层防线（缺一层就等于没有）

| 层 | 机制 | 拦住什么 | 失效后果 |
|---|---|---|---|
| L1 身份 | JWT 携带 `tenant_id`，网关校验签名与过期 | 伪造租户 | 全盘失守 |
| L2 入口 | HTTP header `X-Tenant-Id` 必须与 body 一致，否则 403 | 参数篡改 | 越权写入 |
| L3 数据访问 | 禁止裸 SQL，一律经 `TenantQueryBuilder`，自动注入且不可覆盖 | 漏写 WHERE | 跨租户读 |
| L4 结果断言 | 返回前校验每行 `tenant_id` 与上下文一致，不一致抛异常并告警 | 注入绕过 | 静默泄露 |
| L5 测试 | CI 中跑越权用例，必须 100% 被拒 | 回归 | 上线即失守 |

## 3. 表设计规则（已体现在 001_schema.sql）

```
-- ■■ 1■tenant_id ■■■■■■
-- ■■ 2■■■■■■■■ tenant_id
PRIMARY KEY (tenant_id, diag_id)

-- ■■ 3■■■■■■■■■ tenant_id ■■■■■■■■■■■■■■■■■■■■■■■
KEY idx_asset_time (tenant_id, asset_id, created_at)

-- ■■ 4■■■■■■■■■■■■■■■■■ JSON ■
level VARCHAR(8) NOT NULL   -- ■■■ contributions->>'$.level'

-- ■■ 5■JSON ■■■■■■■■■■■■
contributions JSON NOT NULL -- ■■■■■■
```

反例（禁止）：

```
SELECT * FROM diagnosis WHERE asset_id = 'KA-101';          -- ■ tenant_id
SELECT * FROM diagnosis WHERE JSON_EXTRACT(hypotheses,'$.code')='BEARING_BPFO';  -- JSON ■■■■■■■
```

## 4. TenantQueryBuilder 参考实现（Python）

```
from __future__ import annotations
from dataclasses import dataclass, field

class CrossTenantLeak(RuntimeError):
```

```
    """■■■■■■■■■■■■■■■■■■■■■■"""

@dataclass
class TenantQueryBuilder:
    tenant_id: str
    _table: str = ""
    _where: list[str] = field(default_factory=list)
    _params: list = field(default_factory=list)
    _limit: int = 100

    def table(self, name: str) -> "TenantQueryBuilder":
        if not name.replace("_", "").isalnum():
            raise ValueError("invalid table name")
        self._table = name
        return self

    def where(self, clause: str, *params) -> "TenantQueryBuilder":
        # ■■■■■■■■ tenant_id■■■■■■
        if "tenant_id" in clause:
            raise ValueError("tenant_id is managed by the builder")
        self._where.append(f"({clause})")
        self._params.extend(params)
        return self

    def limit(self, n: int) -> "TenantQueryBuilder":
        self._limit = min(n, 1000)
        return self

    def sql(self) -> tuple[str, list]:
        conds = ["tenant_id = %s", *self._where]
        q = f"SELECT * FROM {self._table} WHERE " + " AND ".join(conds)
        q += f" LIMIT {self._limit}"
        return q, [self.tenant_id, *self._params]

    def fetch(self, conn) -> list[dict]:
        q, p = self.sql()
        with conn.cursor() as cur:
            cur.execute(q, p)
            rows = cur.fetchall()
        # L4■■■■■■
        for r in rows:
            if r.get("tenant_id") != self.tenant_id:
                raise CrossTenantLeak(
                    f"tenant leak: expected {self.tenant_id}, got {r.get('tenant_id')}"
                )
        return rows
```

关键设计点：

- `where()` 里出现 `tenant_id` 直接抛错，防止调用方覆盖
- `fetch()` 逐行断言，宁可抛异常也不返回脏数据
- `limit` 有硬上限，防止全表扫拖垮 TiDB

---

## 5. 越权测试（CI 必跑）

```
# tests/test_tenant_isolation.py
import pytest
from app.tenant import TenantQueryBuilder, CrossTenantLeak

def test_cannot_override_tenant_in_where():
    with pytest.raises(ValueError):
        TenantQueryBuilder("site-a").table("diagnosis").where("tenant_id = 'site-b'")

def test_sql_always_contains_tenant():
    q, p = TenantQueryBuilder("site-a").table("diagnosis").where("asset_id = %s", "KA-101").sql()
    assert "tenant_id = %s" in q
    assert p[0] == "site-a"

def test_cross_tenant_row_raises():
    """■■■■■■■■■■■■■■■■■■/■■■■■■■■■■■■"""
    class FakeCursor:
        def execute(self, *a): pass
        def fetchall(self): return [{"tenant_id": "site-b", "diag_id": "x"}]
        def __enter__(self): return self
```

```
        def __exit__(self, *a): pass
    class FakeConn:
        def cursor(self): return FakeCursor()
    with pytest.raises(CrossTenantLeak):
        TenantQueryBuilder("site-a").table("diagnosis").fetch(FakeConn())

def test_api_rejects_tenant_mismatch(client):
    r = client.post("/v1/diagnose",
                    json={"tenant_id": "site-a", "asset_id": "KA-101", "metrics": {"vib_rms_mm_s": 2.0}},
                    headers={"X-Tenant-Id": "site-b"})
    assert r.status_code == 403
```

> 这四条用例已纳入门禁。任何一条被删除或跳过，视为破坏隔离，PR 直接拒绝。

---

## 6. 运维侧要求

| 项 | 要求 |
|---|---|
| 数据库账号 | 每个服务一个账号，只授必要库表；禁止应用使用 root |
| 账号权限 | 生产账号无 DDL 权限，DDL 走发布流水线 |
| 审计 | 慢查询日志 + 异常查询（无 tenant_id 条件）自动告警 |
| 备份 | BR 全备每日 + 日志备份，RPO ≤ 5 min（DECISIONS P5） |
| 巡检 | 每周核对各租户行数分布，异常增长即排查 |

---

## 7. 如果未来要回退到 PG

`docs/17 §8.1` 已写明：若 TiKV OOM 或容量问题在 Ph1 内无法解决，自动回退为 PG16 主库（原生 RLS）+ TiDB 只读。

回退成本评估：

- DDL 需改写（MySQL → PG 语法），约 2 人日
- `TenantQueryBuilder` 可保留（多一层防御不冲突）
- 服务层零改动（通过 repository 接口隔离）

因此 repository 层必须从一开始就抽象，不允许业务代码直接写 SQL。 这是回退可行性的前提。